Technical Documentation: EasyLog Library

1. Introduction

The **EasyLog** library is a lightweight, thread-safe logging utility designed for .NET applications. It provides a centralized system to track operations, errors, and data transfers by generating structured **JSON or a XML** log files on a daily basis.

2. Architecture

Singleton Pattern

The library uses a thread-safe **Singleton** pattern to ensure that only one instance of the Logger exists during the application's lifecycle. This prevents file access conflicts and optimizes resource usage.

File System Organization

Logs are automatically stored in a **Logs** directory located within the project root.

- **Format**: JSON (System.Text.json) or a XML (System.Xml.Linq)
- **Naming Convention**: yyyy-MM-dd.json or yyyy-MM-dd.Xml (one file per day).
- **Initialization**: The directory is created automatically upon first use.

3. Class Reference

LogEntry Class

This class serves as the data model for log information.

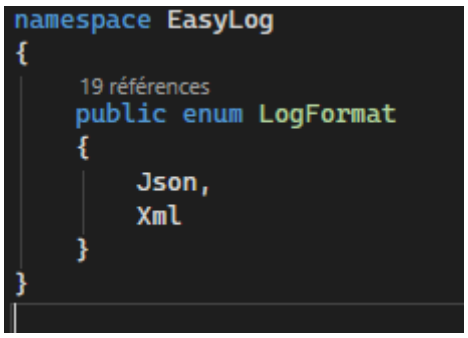
- **time** (string): The timestamp of the event.
- **operationName** (string): Description of the action performed.
- **savetype** (string): Name of the save.
- **sourcePath** (string): The source file or directory path.
- **destinationPath** (string): The destination file or directory path.

- **sizeFile** (long): Size of the data processed (in bytes).
- **timeTransfer** (double): Time taken to complete the operation.
- **encryptionTimeMs (double):** Time taken to encrypt the file in milliseconds.
- **success_Error** (string): Status message or error details.
- **formatJsonOrXml (LogFormat):** The target serialization format for the log entry.

LogFormat Enum

Defines the available output formats for logs :

- **Json:** Standard indented JSON serialization.
- **Xml:** Structured XML.



Logger Class

The main engine responsible for writing logs to the disk.

- **Logger.Instance**: Static property used to access the unique logger instance.
- **WriteLog(LogEntry entry):** Serializes the entry to **JSON** or **XML** format and saves it to the daily log file. The method handles the specific formatting logic—either appending JSON strings or managing XML nodes within a root element—and uses a lock mechanism to ensure thread safety during all file operations.

4. Implementation & Usage

Integration Example

To log an event, you can wrap the call in a helper method as shown below:

Method Definition:

```
public void LogAction(string operation, string name, string source, string destination, long size, double time, double crypttime, string successOrError)
{
    LogFormat formatToUse = settings.logType == LogFormat.Xml ? LogFormat.Xml : LogFormat.Json;
    LogEntry logEntry = new LogEntry
    {
        operationName = operation,
        savetype = name,
        sourcePath = source,
        destinationPath = destination,
        sizeFile = size,
        timeTransfer = time,
        encryptionTimeMs = crypttime,
        success_Error = successOrError,
        formatJsonOrXml = formatToUse
    };
    try
    {
        Logger.Instance.WriteLog(logEntry);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Daily log : Error during {operation} : {ex.Message}");
    }
}
```

Practical Call

Example of how to trigger a log entry during a job execution:

```
LogAction("ExecuteJob : " + job.name, "None", job.sourcePath, job.destinationPath, tailleOctetsSuccess, timeSuccess, encryptionTimeMs, "[Success] Job Executed.");
```

5. Technical Specifications

- **Thread Safety**: Implemented via lock to support multi-threaded environments.
- **Path Discovery**: Dynamically locates the project root folder to ensure logs are stored correctly regardless of the execution context.